



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

O FUNCȚIE HASH PARALELIZABILĂ BAZATĂ PE SISTEME DINAMICE DISCRETE HAOTICE

Absolvent

Aelenei Alex-Ioan

Coordonator științific

Conf. dr. Radu Boriga

București, iunie 2026

Rezumat

Funcțiile hash criptografice reprezintă o componentă fundamentală a securității informatice moderne, însă cele consacrate sunt în mare parte secvențiale și nu valorifică procesoarele moderne cu mai multe nuclee. Această lucrare propune o funcție hash criptografică paralelizabilă, bazată pe sisteme dinamice discrete haotice, care urmărește să ofere un nivel de securitate ridicat fără a sacrifica eficiența. Funcția combină structura Sponge cu un arbore Merkle, permițând paralelizarea eficientă a calculului hash-ului pe sisteme cu mai multe nuclee. Proprietățile criptografice sunt evaluate printr-o analiză statistică detaliată, ce cuprinde teste de confuzie și difuzie, teste de rezistență la coliziuni și analiza distribuției valorilor hash, iar performanța este măsurată pe trei arhitecturi distincte. Rezultatele arată că funcția propusă atinge valori foarte apropiate de cele ideale din punctul de vedere al confuziei și difuziei. De asemenea, funcția prezintă o rezistență la coliziuni comparabilă cu cea a funcțiilor hash consacrate. În plus, datorită arborelui Merkle, viteza de procesare crește aproape liniar cu numărul de nuclee, funcția atingând viteze de ordinul sutelor de MB/s pe sistemele moderne. Astfel, funcția propusă demonstrează că un nivel de securitate adecvat aplicațiilor criptografice actuale poate fi combinat cu o eficiență ridicată pe arhitecturi multi-core.

Cuprins

1	Introducere	5
2	Preliminarii	8
2.1	Funcția hash	8
2.2	Sistemul dinamic	8
2.3	Construcția Sponge	9
2.4	Arborele Merkle	10
3	Funcția haotică	11
3.1	Exponentul Lyapunov	11
3.1.1	Liniarizarea prin matricea jacobiană	11
3.1.2	Spectrul Lyapunov al unui sistem bidimensional	12
3.1.3	Estimarea numerică	13
3.2	Baker's map	14
3.3	Gingerbreadman map	14
3.4	Suprapunerea celor două funcții	15
3.5	Spectrul Lyapunov al funcțiilor	15
4	Funcția hash	19
4.1	Preprocesarea	19
4.2	Generarea frunzelor	19
4.3	Construcția Sponge	20
4.4	Construirea arborelui Merkle	20
5	Analiza performanței	23
5.1	Sensibilitatea la mesaj	23
5.2	Confuzie și difuzie	25
5.3	Rezistența la coliziuni	27
5.4	Distribuția valorilor hash	29
5.5	Flexibilitate	30

5.6	Eficiența computațională	31
5.7	Comparația cu alte funcții hash	34
6	Concluzii	37
	Referințe	38

1 Introducere

În societatea contemporană, influența dezvoltărilor mijloacelor de calcul este incontestabilă și se resfrânge asupra tuturor aspectelor vieții. Acest impact semnificativ nu este unul întâmplător, ci rezultatul a decenii de inovații. Două arii conexe care s-au remarcat timpuriu a fi semnificative în această evoluție au fost criptografia și criptanaliza.

Asemănător cu orice alt domeniu, pe parcursul timpului, în criptografie, au apărut o serie de provocări și puncte vulnerabile, dintre acestea evidențiindu-se: autentificarea și integritatea datelor. Autentificarea asigură veridicitatea identității iar integritatea datelor garantează absența modificărilor nedorite. În acest context, se remarcă funcțiile hash, un instrument criptografic cu aplicații variate. Însăși definiția funcției hash demonstrează caracterul versatil al utilității sale: transformă un șir de lungime arbitrară într-un șir de lungime fixă. Unidirecționalitatea funcției hash determină practicalitatea acesteia. Începând cu sfârșitul celui de-al doilea mileniu, numeroase structuri de funcții hash au fost propuse și analizate. Prima generație de funcții hash moderne a fost reprezentată de MD5 [17], bazată pe construcția Merkle-Damgård [14, 8]. Vulnerabilitățile acestei funcții la atacurile cu preimage sau extinderea lungimii mesajului au devenit rapid evidente. În consecință, au urmat două alte iterații menite să adreseze aceste neajunsuri: SHA-1 [15] și SHA-2 [16]. SHA-1 a fost ulterior compromisă, dar SHA-2 rămâne în continuare standardul la nivel de industrie. În ciuda adoptării generalizate, SHA-2 este fundamentată pe aceeași construcție Merkle-Damgård. Defectele structurale ale acestei construcții au fost relevate de timp. Construcția s-a dovedit vulnerabilă la o multitudine de atacuri, inclusiv cele cu preimage și extinderea lungimii mesajului. Având în vedere acest context, apariția SHA-3 a fost inevitabilă. Structura SHA-3 se îndepărtează de structura definită de Merkle și Damgård, și adoptă construcția Sponge [6].

În procesul de confuzie și difuzie caracteristic funcțiilor hash [18], se remarcă numeroase structuri și construcții. Majoritatea funcțiilor hash moderne se bazează pe substituții, transpoziții, permutări și operații aritmetice. Aceste metode sunt eficiente, însă propri-

etățile lor criptografice nu sunt garantate de un model matematic unitar. Mularea pe o serie de criterii statistice, fără a exista o garanție structurală produce un efect de avalanșă incert, întrucât implică o serie de ipoteze și presupuneri care, în eventualitatea unei vulnerabilități, pot fi invalidate sau propagate la dependenți. Un instrument care conferă prin definiție confuzie și difuzie este reprezentat de sistemele dinamice haotice [2], o subcategorie a sistemelor dinamice discrete. Un astfel de sistem descrie o transformare deterministă care pare complet predictibilă la o primă inspecție, însă, în realitate, manifestă un comportament complex și aparent aleator, caracterizat de o sensibilitate extremă la condițiile inițiale. Aceste sisteme prezintă trei proprietăți definitorii care se aliniază remarcabil cu cerințele criptografice formulate de Shannon [18, 9, 2]. Prima este tranzitivitatea topologică, iar a doua este densitatea orbitelor periodice. Cea de a treia proprietate, sensibilitatea la condițiile inițiale, este asigurată de primele două proprietăți [4]. Astfel, faptul că securitatea sistemelor haotice nu se întemeiază pe nicio construcție euristică este esențial. Absența unei structuri algebrice exploatabile, asociată de regulă cu vulnerabilitatea la atacurile cuantice, transformă această aparentă lipsă de structură într-un atu de securitate.

Având în vedere aceste considerente, lucrarea de față propune o nouă funcție hash bazată pe sisteme dinamice discrete haotice. Securitatea acestei funcții hash se bazează pe proprietățile sistemelor haotice îmbinate cu tehnicile clasice de confuzie și difuzie. Ce diferențiază această funcție hash de cele existente este structura optimizată pentru procesare paralelă. Folosindu-se de arbori Merkle și de construcția Sponge, această funcție hash are o performanță superioară în comparație cu alte funcții similare. Valorificând sistemele hardware moderne multicore, această funcție are o viteză de procesare care o face utilizabilă în scenarii de producție.

În continuare, în capitolul 2 va fi prezentată o serie de definiții și concepte preliminare. În capitolul 3 vor fi prezentate sistemele haotice utilizate în această lucrare și suprapunerea acestora. Capitolul 4 va include o descriere completă a arhitecturii funcției hash propuse, împreună cu o implementare în pseudocod a funcției. În capitolul 5 va fi prezentată

atât analiza criptografică a funcției hash, cât și analiza performanței acesteia, mai ales în comparație cu alte funcții hash similare. În final, în capitolul 6 vor fi prezentate concluziile și viitoare direcții de dezvoltare.

2 Preliminarii

2.1 Funcția hash

O funcție hash criptografică este o transformare deterministă $h: \{0, 1\}^* \rightarrow \{0, 1\}^n$ care asociază datelor de intrare de lungime arbitrară un hash de lungime fixă n . Pentru a fi utilă în context criptografic, ea trebuie să fie eficientă din punct de vedere computațional și să satisfacă trei proprietăți de securitate [14, 8]:

1. Rezistența la preimagine: dat fiind un hash y , este nefezabil să se găsească un mesaj m cu $h(m) = y$.
2. Rezistența la a doua preimagine: dat un mesaj m , este nefezabil să se găsească $m' \neq m$ cu $h(m') = h(m)$.
3. Rezistența la coliziuni: este nefezabil să se găsească orice pereche $m \neq m'$ cu $h(m) = h(m')$.

2.2 Sistemul dinamic

Un sistem dinamic discret este un triplet (T, X, f) , unde:

1. T este spațiul timpului, iar $(T, +)$ este un monoid aditiv
2. X este spațiul de stări sau spațiul fazelor
3. $f: T \times X \rightarrow X$ este funcția de evoluție, care satisface următoarele condiții:

(a) $f(0, x) = x$

(b) $f(t + s, x) = f(t, f(s, x))$ pentru orice $t, s \in T$ și $x \in X$

Devaney [9] definește un sistem haotic ca având trei proprietăți esențiale: sensibilitate la condițiile inițiale, tranzitivitate topologică și faptul că mulțimea orbitelor periodice este densă. Banks [4] demonstrează că tranzitivitatea topologică și densitatea orbitelor periodice implică sensibilitatea la condițiile inițiale.

Exponentul Lyapunov măsoară rata medie de separare exponențială a două orbite pornite din condiții inițiale infinitezimale de apropiate. Pentru un sistem unidimensional cu funcția de evoluție f , el este definit ca:

$$\lambda = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{k=0}^{N-1} \ln|f'(x_k)|, \quad (1)$$

unde x_k sunt iteratele orbitei. O valoare $\lambda > 0$ indică divergența exponențială a orbitelor vecine și constituie semnătura cantitativă a comportamentului haotic, în timp ce $\lambda \leq 0$ corespunde unei dinamici stabile sau cvasi-periodice [9, 2].

Relevanța exponentului Lyapunov pentru o funcție hash reiese tocmai din această sensibilitate la condițiile inițiale. Un exponent strict pozitiv garantează că modificarea unui singur bit din datele de intrare se propagă exponențial pe parcursul iterațiilor, separând rapid orbitele corespunzătoare a două mesaje aproape identice. Acest mecanism stă la baza efectului de avalanșă dorit, prin care date de intrare cu diferențe minime produc hash-uri complet necorelate, iar mărimea exponentului oferă o măsură cantitativă a vitezei cu care difuzia atinge saturația [2].

2.3 Construcția Sponge

Construcția Sponge este o schemă de proiectare a funcțiilor hash care operează asupra unei stări interne de lățime b biți. Starea internă este împărțită într-o rată r și o capacitate c . Asupra stării interne se aplică în mod repetat o permutare fixă f [6]. Procesarea se desfășoară în două faze: în faza de absorbție, blocurile mesajului, de câte r biți, sunt XOR-ate cu partea de rată a stării. Ulterior se aplică permutarea f . În faza de stocare, biții pentru hash sunt extrași din partea de rată, intercalați cu aplicări succesive ale lui f , până la obținerea lungimii dorite a hash-ului. Capacitatea c , care nu este expusă niciodată direct la intrare sau la ieșire, determină nivelul de securitate al construcției.

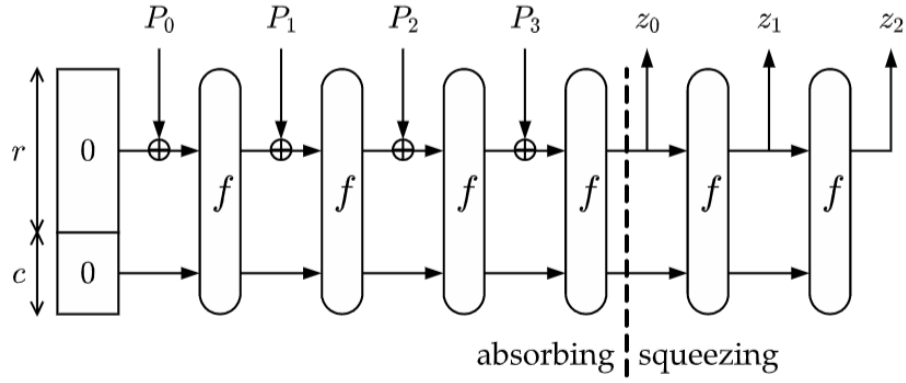


Figura 1: Construcția Sponge [6].

2.4 Arborele Merkle

Arborele Merkle este o structură de date arborescentă, de regulă binară, în care fiecare frunză conține hash-ul unui bloc de date. Fiecare nod intern conține hash-ul obținut prin concatenarea și aplicarea funcției hash asupra hash-urilor copiilor săi [14]. Hash-ul nodului rădăcină, numit rădăcină Merkle, sintetizează astfel întregul set de date într-o singură valoare de lungime fixă. Orice modificare a unui bloc se propagă în mod inevitabil până la rădăcină. Această organizare permite paralelizarea funcțiilor hash, întrucât subarborii independenți pot fi procesați simultan.

3 Funcția haotică

Nucleul funcției hash propuse îl constituie o transformare haotică aplicată repetat asupra stării interne. Această transformare nu se reduce la o singură funcție, ci la suprapunerea a două sisteme dinamice distincte, alese astfel încât slăbiciunile fiecăruia să fie compensate de celălalt. Prima este baker's map, responsabilă de difuzie, iar a doua este gingerbreadman map, responsabilă de confuzie. Compunerea lor produce, la fiecare iterație, atât amestecarea uniformă a stării, cât și o neliniaritate suficientă pentru a împiedica orice descriere algebrică simplă a procesului.

3.1 Exponentul Lyapunov

Afirmația că o funcție manifestă un comportament haotic poate fi cuantificată riguros prin intermediul exponentului Lyapunov. Acesta măsoară rata medie cu care două traiectorii pornite din condiții inițiale infinit de apropiate se îndepărtează una de cealaltă. Fie două puncte de start separate de o perturbație inițială δ_0 . Pentru un sistem haotic, distanța dintre traiectoriile lor crește, în medie, exponențial cu numărul de iterații n :

$$|\delta_n| \approx |\delta_0| e^{\lambda n} \quad (2)$$

Coeficientul λ din exponent este tocmai exponentul Lyapunov. O valoare pozitivă, $\lambda > 0$, indică divergența exponențială a traiectoriilor și, implicit, sensibilitatea extremă la condițiile inițiale specifică haosului. O valoare negativă indică, dimpotrivă, convergența traiectoriilor către o orbită stabilă.

3.1.1 Liniarizarea prin matricea jacobiană

Pentru a măsura această divergență, perturbația δ nu se urmărește prin reaplicarea funcției asupra a două puncte distincte, deoarece, după câțiva pași, distanța dintre ele depășește domeniul în care aproximarea liniară este validă. În schimb, evoluția unei

perturbații infinitezimale este descrisă de liniarizarea funcției în jurul punctului curent al traiectoriei. Fie funcția bidimensională $F(x, y) = (f(x, y), g(x, y))$. Liniarizarea sa în punctul $p_n = (x_n, y_n)$ este dată de matricea jacobiană, care conține derivatele parțiale ale componentelor funcției:

$$J(p_n) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix}_{p_n} \quad (3)$$

Aplicată unui vector tangent v (o perturbație infinitesimală), matricea jacobiană indică modul în care acel vector este rotit și scalat de o singură iterație a funcției. Astfel, evoluția perturbației de-a lungul orbitei se reduce la o înmulțire matrice-vector la fiecare pas:

$$v_{n+1} = J(p_n) v_n \quad (4)$$

După N iterații, vectorul tangent inițial v_0 este transformat prin produsul matricelor jacobiene evaluate succesiv în lungul traiectoriei:

$$v_N = J(p_{N-1}) J(p_{N-2}) \cdots J(p_0) v_0 \quad (5)$$

Exponentul Lyapunov se obține ca rata medie de creștere logaritmică a normei vectorului tangent, pe măsură ce numărul de pași tinde la infinit:

$$\lambda = \lim_{N \rightarrow \infty} \frac{1}{N} \ln \frac{|v_N|}{|v_0|} \quad (6)$$

3.1.2 Spectrul Lyapunov al unui sistem bidimensional

Un sistem bidimensional nu posedă un singur exponent, ci un spectru de doi exponenți, $\lambda_1 \geq \lambda_2$, câte unul pentru fiecare direcție independentă a spațiului de stări. Aceștia descriu deformarea unui disc infinitesimal de condiții inițiale: λ_1 caracterizează direcția de întindere maximă, iar λ_2 direcția de contracție. Cel mai mare exponent, λ_1 , domină comportamentul pe termen lung, fiindcă orice vector tangent ales aleatoriu se aliniază

rapid direcției de creștere maximă; pozitivitatea sa este criteriul standard pentru identificarea haosului. Suma celor doi exponenți este legată de variația ariei sub acțiunea funcției, prin logaritmul determinantului jacobian:

$$\lambda_1 + \lambda_2 = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} \ln |\det J(p_n)| \quad (7)$$

Pentru o funcție care conservă aria, precum gingerbreadman map, această sumă este nulă: orice întindere pe o direcție este compensată exact de o contracție pe cealaltă.

3.1.3 Estimarea numerică

Calculul direct al produsului de matrice jacobiene este instabil numeric. Deoarece toți vectorii tangenți tind să se alinieze direcției dominante, al doilea exponent devine imposibil de recuperat, iar normele cresc sau scad necontrolat, conducând la depășiri de domeniu. Soluția standard este algoritmul lui Benettin [5], care propagă simultan doi vectori tangenți ortonormali și îi re-ortonormalizează periodic prin procedeul Gram-Schmidt. La fiecare pas, înainte de re-ortonormalizare, se acumulează logaritmul factorilor de creștere ai celor doi vectori; mediile acestor logaritmi, raportate la numărul de pași, converg către cei doi exponenți Lyapunov.

Algoritmul 1: Estimarea spectrului Lyapunov [5]

Date de intrare: Funcția F , jacobianul J , punctul inițial p_0 , numărul de pași N

Rezultat: Exponenții Lyapunov λ_1, λ_2

```
1 inițializează  $v_1 \leftarrow (1, 0)$ ,  $v_2 \leftarrow (0, 1)$ ;  
2 inițializează  $s_1 \leftarrow 0$ ,  $s_2 \leftarrow 0$ ;  
3 pentru  $n \leftarrow 0$  până la  $N - 1$  execută  
4    $v_1 \leftarrow J(p_n) v_1$ ;  
5    $v_2 \leftarrow J(p_n) v_2$ ;  
6    $r_1 \leftarrow |v_1|$ ;  $v_1 \leftarrow v_1 / r_1$ ;  
7    $v_2 \leftarrow v_2 - (v_2 \cdot v_1) v_1$ ;  
8    $r_2 \leftarrow |v_2|$ ;  $v_2 \leftarrow v_2 / r_2$ ;  
9    $s_1 \leftarrow s_1 + \ln r_1$ ;  $s_2 \leftarrow s_2 + \ln r_2$ ;  
10   $p_{n+1} \leftarrow F(p_n)$ ;  
11  $\lambda_1 \leftarrow s_1 / N$ ;  $\lambda_2 \leftarrow s_2 / N$ ;
```

3.2 Baker's map

Baker's map este un sistem dinamic clasic [9], a cărui denumire provine dintr-o analogie sugestivă cu procesul de frământare a aluatului. Brutarul întinde aluatul pe o direcție, dublându-i lungimea, după care îl pliază la loc peste sine. Repetarea acestui gest amestecă rapid și ireversibil orice particulă din interiorul aluatului, dispersând-o în întreaga masă. Formal, sistemul dinamic are următoarea relație de recurență:

$$(x_{n+1}, y_{n+1}) = \begin{cases} (2x_n, \frac{y_n}{2}) & \text{dacă } 0 \leq x_n < \frac{1}{2} \\ (2 - 2x_n, 1 - \frac{y_n}{2}) & \text{dacă } \frac{1}{2} \leq x_n < 1 \end{cases} \quad (8)$$

3.3 Gingerbreadman map

Gingerbreadman map, introdusă de Devaney [9], este un sistem dinamic haotic bidimensional. Numele său provine din forma caracteristică a atractorului, care, vizualizat în plan, amintește de silueta unei figurine din turtă dulce. Definitorie pentru această funcție

este prezența valorii absolute, singura sursă de neliniaritate.

Formal, sistemul dinamic are următoarea relație de recurență:

$$(x_{n+1}, y_{n+1}) = (1 - y_n + |x_n|, x_n) \quad (9)$$

3.4 Suprapunerea celor două funcții

Cele două sisteme se completează reciproc tocmai prin slăbiciunile lor complementare. Baker's map oferă o amestecare uniformă și lipsită de insule de stabilitate, dar este liniară. Gingerbreadman map oferă neliniaritatea absentă din prima, dar conține regiuni cvasi-periodice. Aplicată prima, baker's map dispersează punctul pe întregul spațiu de stări. Astfel, dacă punctul curent se află într-o regiune cvasi-periodică a gingerbreadman map, punctul va fi împins într-o nouă orbită. Aplicată în continuare, gingerbreadman map împletește neliniar coordonatele astfel decorelate. În consecință, această rundă compusă, deși complet deterministă, manifestă un comportament haotic, cu o sensibilitate extremă la condițiile inițiale. Alegerea celor două sisteme a fost motivată de faptul că ambele prezintă un comportament haotic și sunt eficiente din punct de vedere computațional. Definițiile acestora presupun o implementare cu operații aritmetice simple, fiind utilizate tipuri de date native.

3.5 Spectrul Lyapunov al funcțiilor

Pentru a confirma empiric comportamentul haotic descris, spectrul Lyapunov a fost estimat numeric pentru fiecare dintre cele trei funcții, folosind algoritmul lui Benettin aplicat pe un număr mare de condiții inițiale alese aleatoriu. Figurile următoare reprezintă cei doi exponenți, λ_1 și λ_2 , codificați cromatic în funcție de valoarea lor, în planul condițiilor inițiale (x, y) .

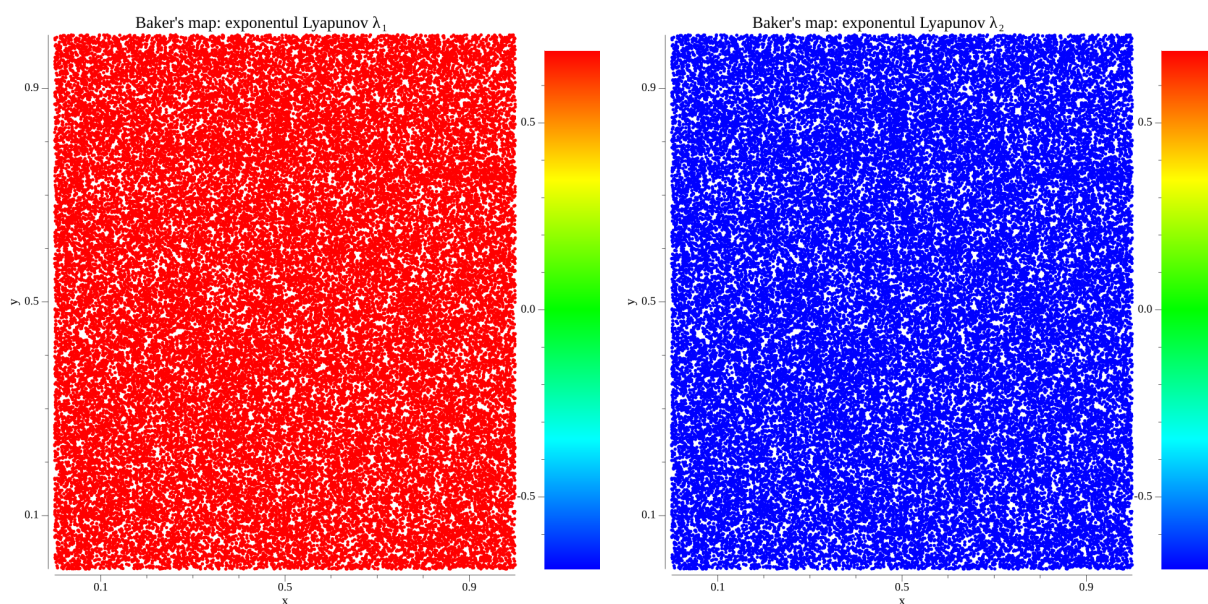


Figura 2: Exponenții Lyapunov λ_1 (stânga) și λ_2 (dreapta) ai baker's map

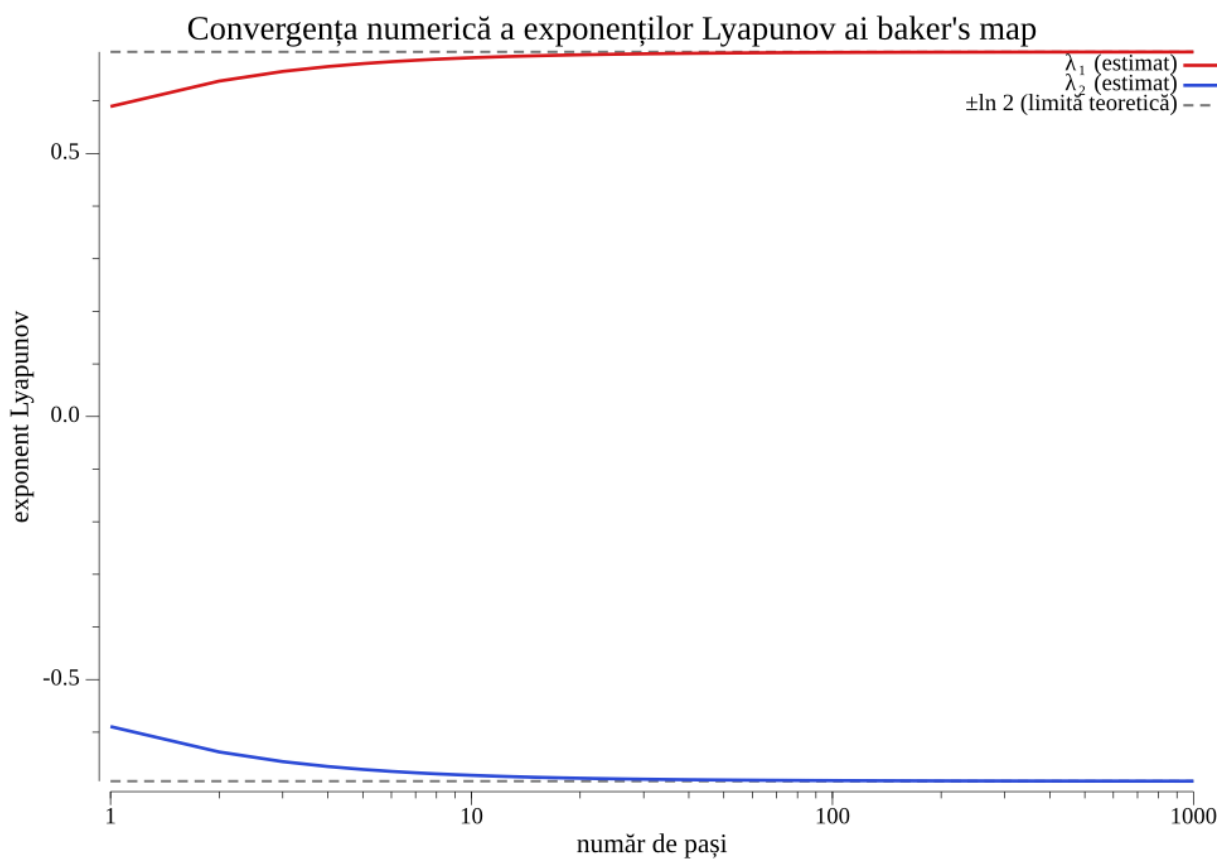


Figura 3: Convergența numerică a exponenților Lyapunov ai baker's map către limitele teoretice $\pm \ln 2$

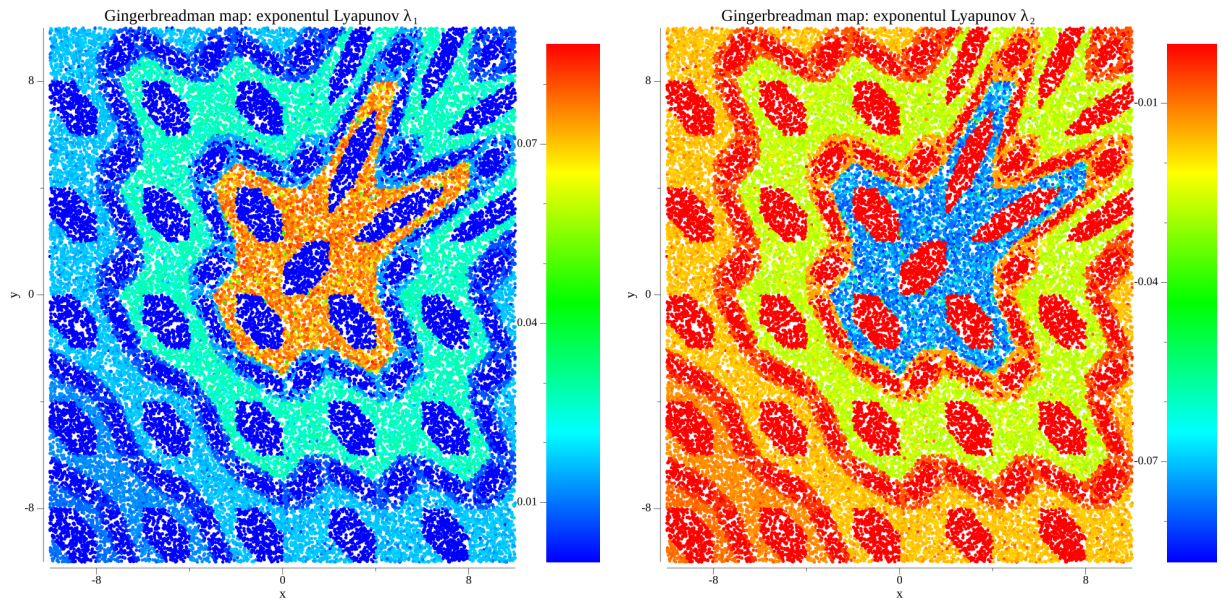


Figura 4: Exponenții Lyapunov λ_1 (stânga) și λ_2 (dreapta) ai gingerbreadman map

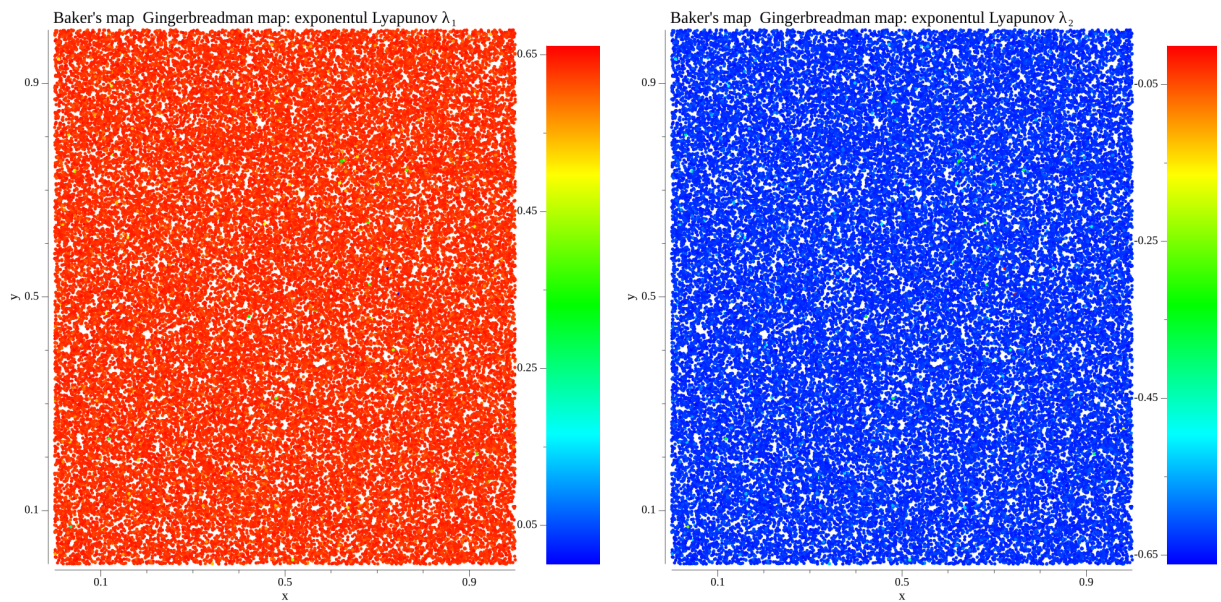


Figura 5: Exponenții Lyapunov λ_1 (stânga) și λ_2 (dreapta) ai rundeii compuse

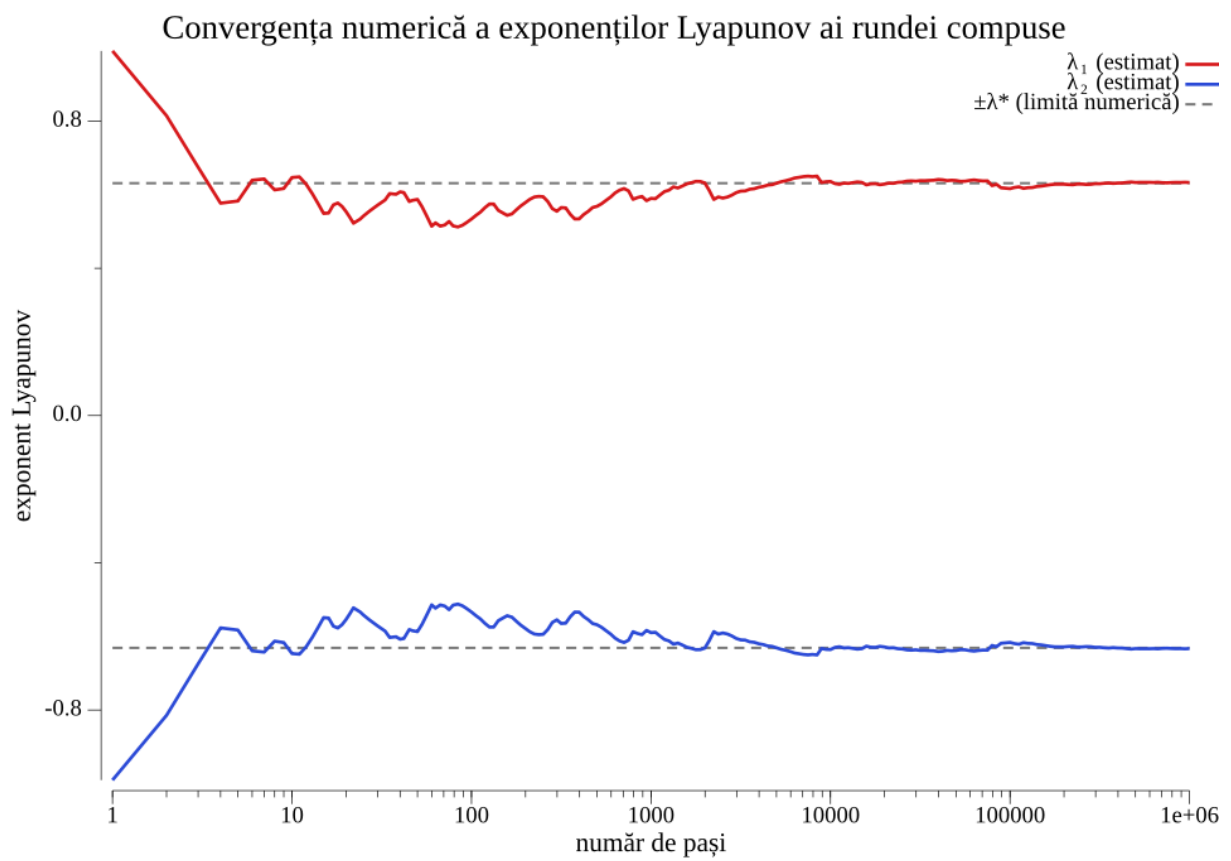


Figura 6: Convergența numerică a exponenților Lyapunov ai runde compuse către limita numerică $\pm\lambda^* \approx \pm 0,6331$

4 Funcția hash

Funcția hash propusă în această lucrare se bazează pe toate conceptele prezentate în secțiunile anterioare. Structura Sponge este aplicată pe blocuri, iar apoi prin intermediul arborelui Merkle se obține hash-ul întregului mesaj. Dimensiunea stării interne este de 1024 biți, împărțită în 512 biți pentru rată și 512 biți de capacitate. Transformarea internă a stării este realizată printr-o funcție haotică, completată de metode clasice de confuzie și difuzie. Pentru fragmentele următoare de pseudocod, considerăm următoarele convenții de notație:

- S reprezintă starea internă, formată din 32 de cuvinte de 32 de biți, S_i fiind cuvântul de pe poziția i .
- $a \oplus b$ reprezintă XOR-are (disjuncție exclusivă) bit cu bit.
- $a \lll k$ reprezintă rotația circulară spre stânga a cuvântului a cu k poziții.
- $a \bmod b$ reprezintă restul împărțirii lui a la b .
- l reprezintă lungimea hash-ului final, exprimată în biți.

4.1 Preprocesarea

Secțiunea de preprocesare are rolul de a pregăti mesajul inițial pentru absorbția în construcția Sponge. Astfel, lungimea mesajului preprocesat trebuie să fie un multiplu al ratei de 512 biți. Considerând că mesajul inițial este M , alcătuit din $M = M_1 \| M_2 \| \dots \| M_n$, unde M_i reprezintă un byte, mesajul preprocesat va fi:

$$M' = M \| 0x80 \| 0x00 \| \dots \| 0x00 \| 0x01, \quad (10)$$

4.2 Generarea frunzelor

După preprocesare, mesajul este împărțit în blocuri de dimensiune configurabilă. Fiecare bloc B_i va reprezenta o frunză în arborele Merkle, iar hash-ul său va fi calculat prin

aplicarea construcției Sponge. Trebuie menționat faptul că dimensiunea unui bloc este un parametru configurabil.

4.3 Construcția Sponge

Construcția Sponge este aplicată pe fiecare bloc B_i pentru a genera hash-ul. Fiecare bloc de 512 biți este XOR-at cu partea de rată a stării interne, după care se aplică funcția haotică. Permutarea f a construcției Sponge este aplicată după fiecare absorbție a unui bloc. Starea internă este împărțită în 16 secțiuni, fiecare formată din două cuvinte de 32 de biți, unul din partea de rată și celălalt din partea de capacitate. Ulterior, cele două sisteme haotice sunt aplicate secvențial pe fiecare secțiune de 20 de ori. După ce toate secțiunile au fost procesate, o etapă de difuzie amestecă secțiunile între ele. Astfel, mai întâi se amestecă cuvintele vecine din fiecare jumătate, apoi se amestecă cele două jumătăți între ele. Amestecarea se realizează prin rotații de biți și XOR-are, pentru a asigura propagarea schimbărilor în întreaga stare internă.

4.4 Construirea arborelui Merkle

Construcția Sponge și algoritmul descris anterior sunt utilizate în cadrul arborelui Merkle. Dacă nodul curent este o frunză, atunci datele absorbite sunt reprezentate de blocurile B_i ale mesajului preprocesat M' . Dacă nodul curent este un nod intern, atunci datele absorbite sunt formate prin concatenarea ratelor de la toți copiii nodului curent. Trebuie menționat faptul că numărul maxim de copii ai unui nod intern este configurabil. La finalul procesului, rădăcina arborelui Merkle conține hash-ul întregului mesaj. Înainte de faza de stoarcere a hash-ului din construcția Sponge, se aplică un număr de runde intermediare, unde nu este absorbit niciun bloc de date. Numărul de runde intermediare n este determinat de lungimea mesajului inițial și de dimensiunea finală a hash-ului prin următoarea formulă:

$$n = 9 + (|M| \bmod 37 + |l| \bmod 31) \bmod 11 \quad (11)$$

Algoritmul 2: Permutarea stării

Date de intrare: starea internă S_i , $i \in \overline{0, 15}$

Rezultat: starea nou permutată S'_i

```
1 pentru  $i \leftarrow 0$  până la 15 execută
2    $x \leftarrow S_i$ ;
3    $y \leftarrow S_{i+16}$ ;
4   pentru  $r \leftarrow 1$  până la 20 execută
5     // baker's map
6     dacă  $x < 2^{31}$  atunci
7        $x \leftarrow x \ll 1$ ;
8        $y \leftarrow y \gg 1$ ;
9     altfel
10       $x \leftarrow (\neg x) \ll 1$ ;
11       $y \leftarrow ((\neg y) \gg 1) \vee 2^{31}$ ;
12    // gingerbreadman map
13    dacă  $x \geq 2^{31}$  atunci
14       $a \leftarrow -x$ ;
15    altfel
16       $a \leftarrow x$ ;
17     $(x, y) \leftarrow (a - y, x)$ ;
18   $S_i \leftarrow x$ ;
19   $S_{i+16} \leftarrow y$ ;
20  // cuplarea coloanelor: reinjectare în capacitatea următoare
21   $t \leftarrow (i + 1) \bmod 16 + 16$ ;
22   $S_t \leftarrow S_t \oplus (x \lll 9)$ ;
23   $S_t \leftarrow S_t \oplus (y \lll 17)$ ;
24  // difuzie: pas înainte și pas înapoi în fiecare jumătate
25  pentru fiecare  $b \in \{0, 16\}$  execută
26    pentru  $i \leftarrow 0$  până la 15 execută
27       $S_{b+i} \leftarrow S_{b+i} \oplus (S_{b+(i+1) \bmod 16} \lll 13)$ ;
28  pentru fiecare  $b \in \{0, 16\}$  execută
29    pentru  $i \leftarrow 15$  până la 0 execută
30       $S_{b+i} \leftarrow S_{b+i} \oplus (S_{b+(i+15) \bmod 16} \lll 11)$ ;
31  // difuzie: pas de amestecare între rată și capacitate
32  pentru  $i \leftarrow 0$  până la 15 execută
33     $S_i \leftarrow S_i \oplus (S_{i+16} \lll 7)$ ;
34  pentru  $i \leftarrow 0$  până la 15 execută
35     $S_{i+16} \leftarrow S_{i+16} \oplus (S_i \lll 19)$ ;
```

La final, hash-ul este extras din partea de rată a stării interne. Se execută faza de stoarcere până când se obțin suficienți biți pentru a forma hash-ul. Având în vedere că rata este de 512 biți, dacă lungimea hash-ului final l nu este multiplu de 512, biții rămași sunt trunchiați.

5 Analiza performanței

În această secțiune, vom analiza performanța funcției hash propuse. Criteriile de analiză vor fi multiple, și va fi inclusă și comparația cu alte funcții propuse. Intenția este aceea de a demonstra că funcția hash propusă îndeplinește toate proprietățile de securitate necesare. Mai mult de atât, vom arăta că funcția hash propusă este și eficientă din punct de vedere computațional spre deosebire de alte funcții hash similare. Pentru a putea efectua o comparație corectă, am ales următorii parametrii pentru testare:

- Dimensiunea hash-ului $l = 128$ biți
- Dimensiunea blocurilor absorbite de construcția Sponge este de 16 kilobytes.
- Numărul maxim de copii în arborele Merkle este de 8.

5.1 Sensibilitatea la mesaj

Pentru a demonstra sensibilitatea funcției hash propuse la modificări minime aplicate asupra mesajului considerăm următorul mesaj inițial:

M0 - "Timpul petrecut cu pisici nu este niciodată irosit."

Ulterior, considerăm 5 variante ale acestui mesaj, fiecare având câte o diferență față de mesajul inițial:

1. M1 - i din "pisici" este schimbat cu a
2. M2 - t din "petrecut" este schimbat cu T
3. M3 - punctul final este schimbat cu spațiu
4. M4 - spațiul după "pisici" este schimbat cu !
5. M5 - i este adăugat la începutul cuvântului "irosit"

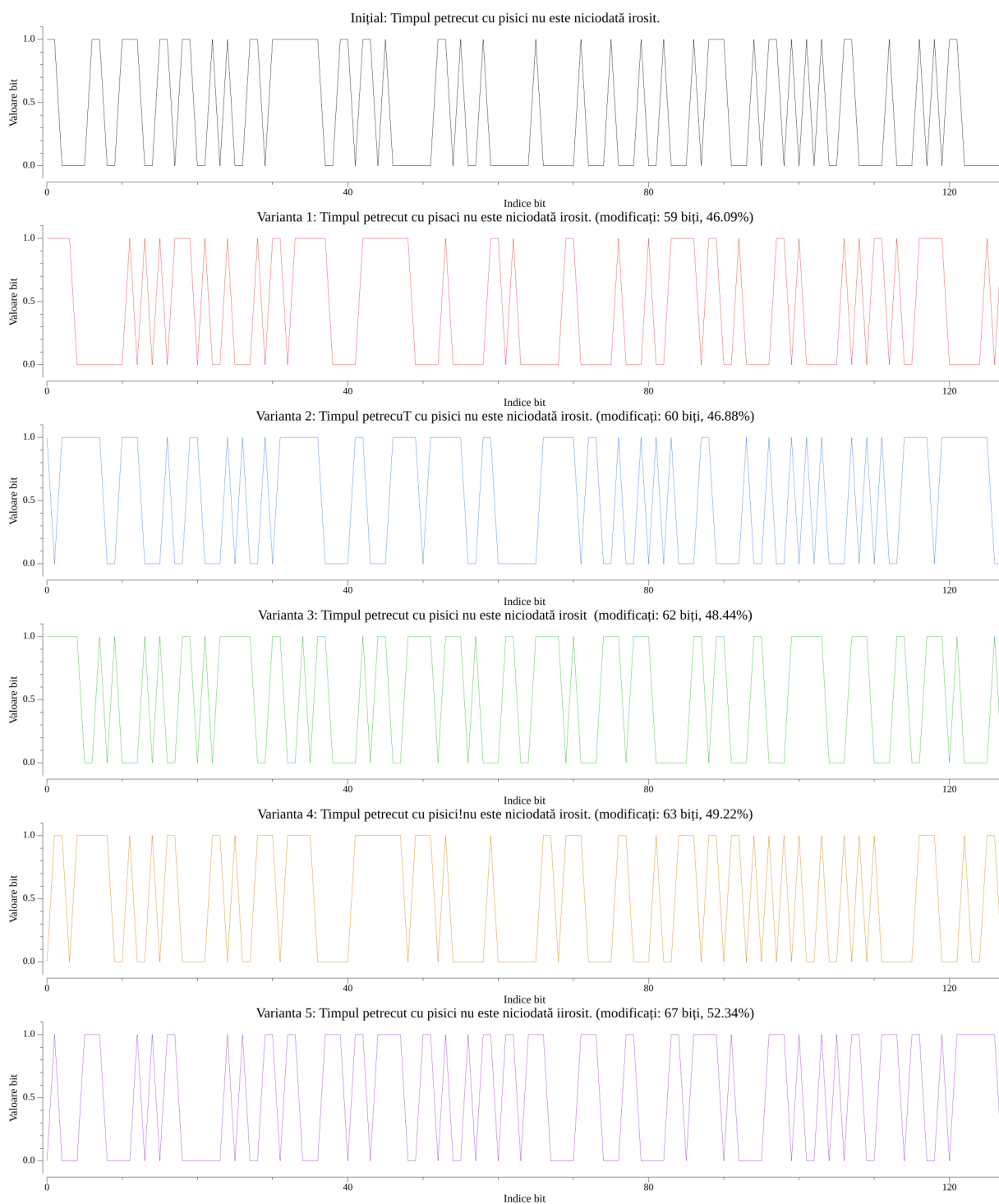


Figura 7: Reprezentarea binară a valorilor hash pentru mesajele M0-M5

Rezultatul funcției hash pentru fiecare mesaj este prezentat în tabelul de mai jos:

Mesaj	Valoarea hash	Biți modificați (%)
M0	c339b29bf9b40d20411122e2d5308ac0	—
M1	f015748b7c3f841a06089ec8682b4f05	46.09
M2	bf3898a5f863df303ec9518495153dfc	46.88
M3	f94535f32c2cf7467a3b83631f1c6742	48.44
M4	6f92c34ef07f7410370c4edaa92a0e26	49.22
M5	470ac0a6c76f34b6e1c61bd0e959d97e	52.34

Tabela 1: Sensibilitatea la mesaj

5.2 Confuzie și difuzie

Confuzia și difuzia sunt două principii fundamentale de proiectare a sistemelor criptografice, introduse de Shannon [18] în contextul sistemelor criptografice. Confuzia urmărește să facă relația dintre intrare și ieșire cât mai complexă și mai greu de analizat. Difuzia urmărește disiparea structurii statistice a intrării asupra întregii ieșiri. Astfel, fiecare bit al ieșirii depinde de cât mai mulți biți ai intrării [18]. Deși au fost formulate inițial pentru cifruri, aceste două criterii au fost adoptate ulterior ca proprietăți esențiale pentru orice primitivă criptografică, inclusiv pentru funcțiile hash. În cazul unei funcții hash, scopul confuziei și al difuziei este de a îndepărta orice corelație statistică dintre mesajul de intrare și valoarea hash rezultată. În consecință, o modificare minimă a mesajului produce o valoare hash aparent fără nicio legătură cu cea inițială.

Modalitatea de testare a funcției hash este următoarea: se generează un set de N mesaje aleatorii, de lungime aleatoare. Pentru fiecare mesaj se calculează hash-ul, se schimbă un bit ales arbitrar din mesajul inițial și se calculează hash-ul și pentru mesajul modificat. Se calculează distanța Hamming dintre cele două hash-uri și se notează această valoare cu $B_i, \forall i \in \overline{1, N}$. Ulterior, se definesc următoarele metrice pentru fiecare test:

1. Numărul mediu de biți modificați:

$$B_{med} = \frac{1}{N} \sum_{i=1}^N B_i \quad (12)$$

2. Numărul minim de biți modificați:

$$B_{\min} = \min \{B_i \mid i \in \overline{1, N}\} \quad (13)$$

3. Numărul maxim de biți modificați:

$$B_{\max} = \max \{B_i \mid i \in \overline{1, N}\} \quad (14)$$

4. Probabilitatea medie de modificare a biților:

$$P = \frac{B_{med}}{l} \cdot 100\% \quad (15)$$

5. Abaterea standard a numărului de biți modificați:

$$\Delta B = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (B_i - B_{med})^2} \quad (16)$$

6. Abaterea standard a probabilității de modificare:

$$\Delta P = \sqrt{\frac{1}{N-1} \sum_{i=1}^N \left(\frac{B_i}{l} - P\right)^2} \times 100\% \quad (17)$$

Rezultatele obținute pentru diferite valori ale numărului de teste N sunt prezentate în tabelul de mai jos:

N	B_{med}	$B_{\min}$	$B_{\max}$	P (%)	ΔB	ΔP (%)
1000	64.0420	46	84	50.0328	5.7768	4.5131
2500	63.9260	43	86	49.9422	5.6045	4.3785
5000	64.0060	44	85	50.0047	5.7832	4.5181
7500	64.0007	43	87	50.0005	5.5848	4.3631
10000	64.0771	42	85	50.0602	5.6502	4.4142

Tabela 2: Statisticile de confuzie și difuzie pentru valori diferite ale numărului de teste N

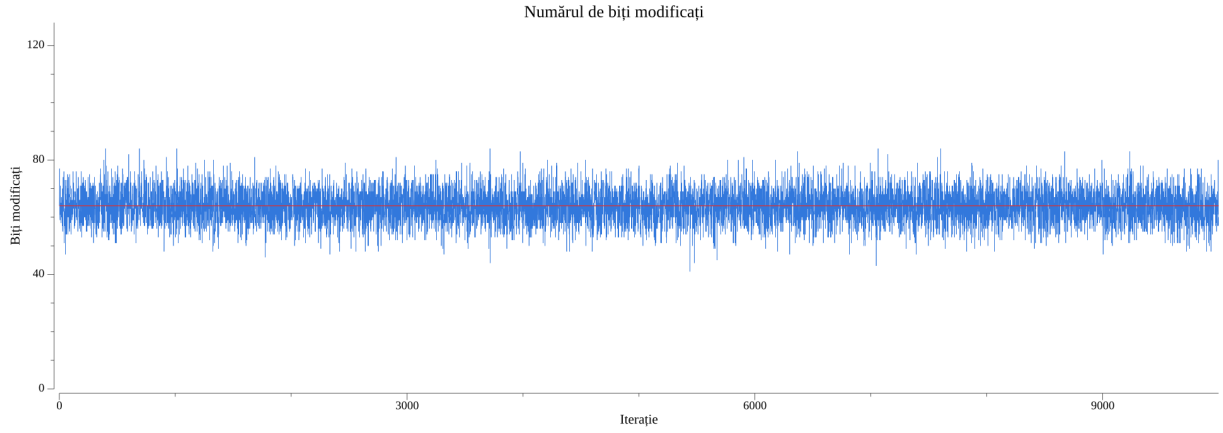


Figura 8: Numărul de biți modificați pentru fiecare B_i , cu $N = 10000$

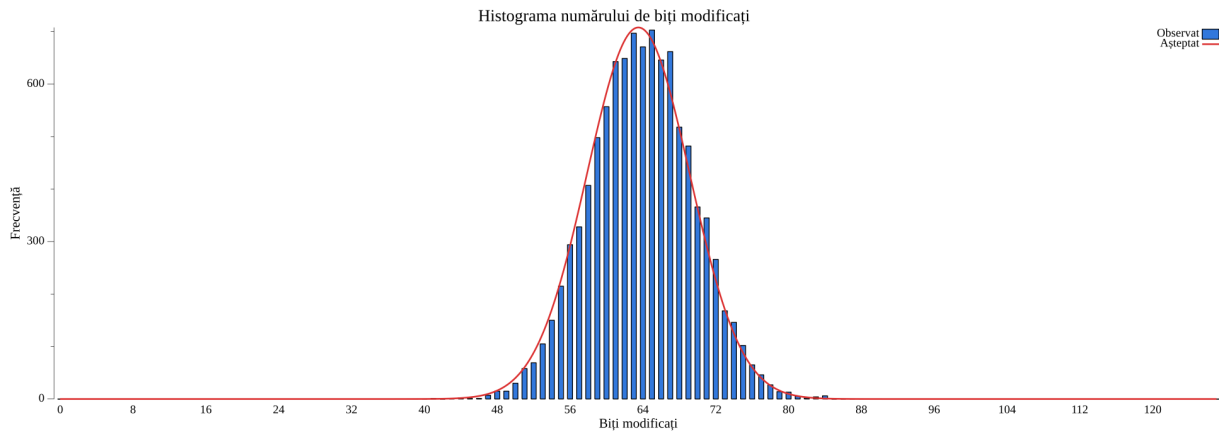


Figura 9: Histograma numărului de biți modificați pentru $N = 10000$

În mod ideal, pentru o funcție hash cu proprietăți de confuzie și difuzie bune, valoarea numărului mediu de biți modificați B_{med} ar trebui să tindă spre $\frac{l}{2}$. Conform rezultatelor obținute, se observă că B_{med} este foarte aproape de 64, ceea ce reprezintă exact jumătate din lungimea hash-ului $l = 128$ biți. Valorile lui ΔB și ΔP indică faptul că numărul de biți modificați este grupat aproape de valoarea medie. Conform histogramei 9, se observă că valorile se încadrează într-o distribuție normală (linia roșie). În mod similar, conform graficului 8, se observă că numărul de biți modificați variază aleator în vecinătatea valorii medii.

5.3 Rezistența la coliziuni

O coliziune apare atunci când două mesaje distincte $M \neq M'$ produc aceeași valoare hash, adică $h(M) = h(M')$. Rezistența la coliziuni este una dintre proprietățile fun-

damentale ale unei funcții hash criptografice. Această caracteristică garantează că este dificil din punct de vedere computațional să se găsească o astfel de pereche de mesaje. Deoarece o funcție hash comprimă mesaje de lungime arbitrară într-o valoare de lungime fixă de l biți, spațiul intrărilor este mult mai mare decât cel al ieșirilor. Conform principiului cutiei lui Dirichlet, existența coliziunilor este, prin urmare, inevitabilă. Rezistența la coliziuni nu înseamnă că funcția hash nu are coliziuni, ci doar că acestea sunt practic imposibil de găsit. Pornind de la paradoxul zilei de naștere, se poate demonstra că, pentru o funcție hash de l biți, sunt necesare aproximativ $2^{l/2}$ operații de calcul al hash-ului pentru a găsi o coliziune. Pentru ca un astfel de atac să rămână nefezabil în practică, lungimea valorii hash trebuie să fie suficient de mare. De aceea, în cazul de față am ales $l = 128$ biți, ceea ce ridică dificultatea atacului la ordinul a 2^{64} operații. Relevanța acestei cerințe este subliniată de rezultatele relativ recente de criptanaliză, care au demonstrat vulnerabilități în funcții hash consacrate, precum MD5 și SHA-1. Întrucât funcțiile hash bazate pe sisteme dinamice haotice operează cu numere reale, o demonstrație matematică riguroasă a rezistenței la coliziuni este dificil de realizat.

Modalitatea de evaluare a rezistenței la coliziuni hash este următoarea: se generează un set de N mesaje aleatorii, de lungime aleatoare. Pentru fiecare mesaj M se calculează hash-ul, se schimbă un bit ales arbitrar din mesajul inițial obținându-se M' . Se calculează hash-ul și pentru mesajul modificat și se calculează numărul de bytes egali între cele două hash-uri. În plus, se calculează și suma diferențelor absolute a bytes-ilor celor două hash-uri după formula:

$$D_i = \sum_{j=1}^{l/8} |Dec(h(M)_j) - Dec(h(M_i)_j)| \quad (18)$$

Considerăm că $Dec(x)$ este conversia din byte în formă zecimală, iar $h(M)_j$ reprezintă al j -lea byte al hash-ului pentru mesajul M . Astfel, este calculată suma minimă, suma maximă, media sumelor și media sumelor raportată la numărul de bytes al hash-ului.

Bytes egali	0	1	2	3	4	5
Hash-uri	9401	589	10	0	0	0
Valori teoretice	9393	590	17	0	0	0

Tabela 3: Distribuția numărului de bytes egali pentru $N = 10000$ mesaje

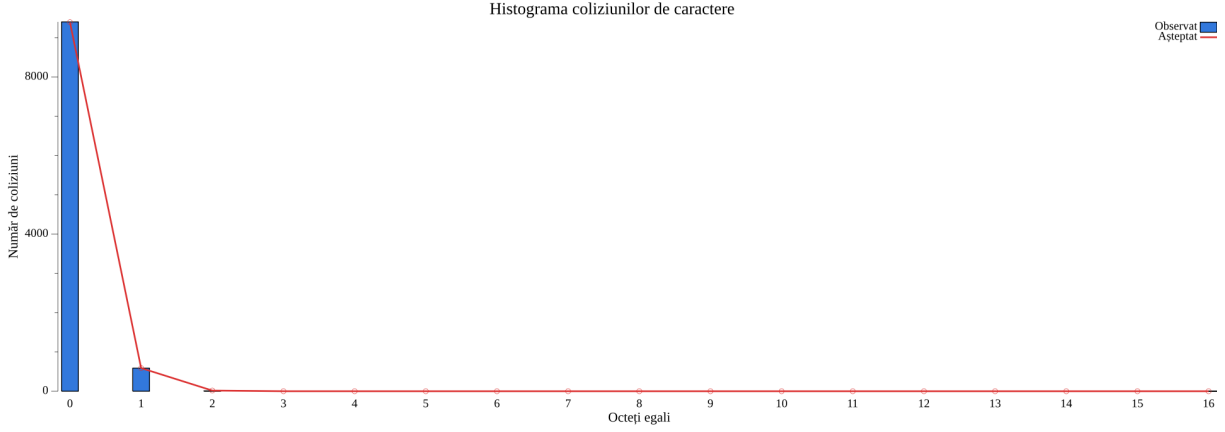


Figura 10: Distribuția numărului de bytes egali pentru $N = 10000$ mesaje

N	Minim	Maxim	Medie	Medie/byte
1000	687	2193	1365.9250	85.3703
5000	592	2234	1365.3423	85.3338
10000	567	2278	1364.9053	85.3066

Tabela 4: Suma diferențelor absolute a bytes-ilor în funcție de N

Valoarea teoretică pentru media sumelor diferențelor absolute a bytes-ilor, pentru un hash de lungime $l = 128$ biți este 1360. Astfel, se observă că media sumelor se apropie foarte mult de această valoare teoretică, ceea ce indică o rezistență foarte bună la coliziuni. În plus, conform tabelului 3 și graficului 10, se observă că numărul de colizuni pentru hash-uri este sub numărul teoretic. În consecință, rezistența la coliziuni a funcției hash propuse este foarte bună.

5.4 Distribuția valorilor hash

Una dintre primele forme de atac asupra unei funcții hash este criptanaliza statistică, care exploatează eventualele corelații dintre mesajul de intrare și valoarea hash rezultată. Pentru a împiedica astfel de atacuri, valorile hash trebuie să fie distribuite cât mai uniform pe întreg spațiul valorilor de ieșire. Distribuția valorilor trebuie să nu păstreze nicio urmă

a structurii statistice a mesajului de intrare. Modalitatea de a testa distribuția valorilor hash este similară cu cea utilizată pentru testarea confuziei și difuziei. Se generează un set de N mesaje aleatorii, de lungime aleatoare. Pentru fiecare mesaj M se calculează hash-ul, se schimbă un bit ales aleatoriu din mesajul inițial obținându-se M' . Se calculează hash-ul și pentru mesajul modificat și se numără ce biți s-au schimbat între $h(M)$ și $h(M')$.

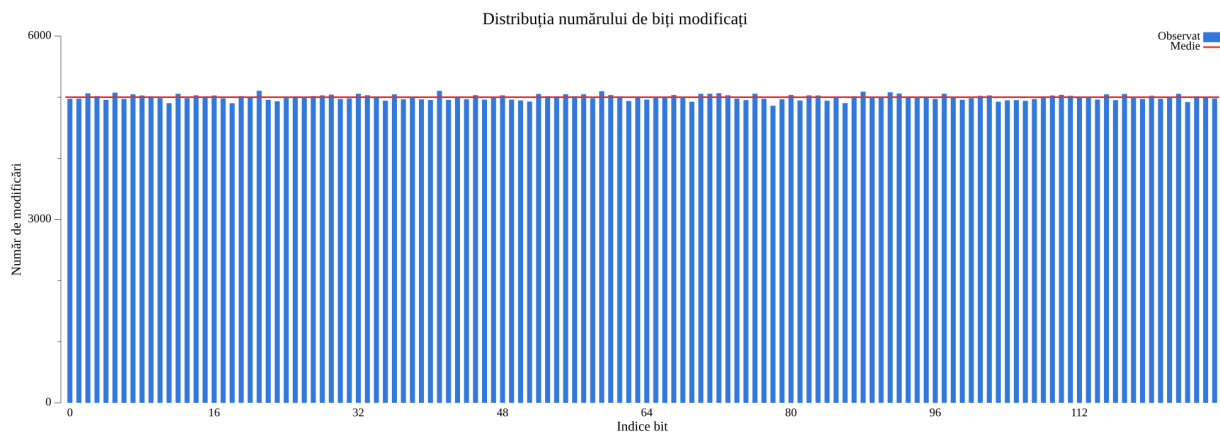


Figura 11: Frecvența modificării fiecărui bit al hash-ului pentru $N = 10000$ mesaje

Valoarea ideală pentru frecvența modificării fiecărui bit al hash-ului este de 50% din numărul total de teste. Astfel, pentru $N = 10000$ mesaje, fiecare bit al hash-ului ar trebui să se schimbe de aproximativ 5000 de ori. Conform graficului 11, se observă că frecvența modificării fiecărui bit al hash-ului se apropie foarte mult de această valoare ideală. Valoarea minimă a frecvenței schimbării unui bit este de 4862, valoarea maximă este de 5107, iar media este de 5000.98. Aceste rezultate indică faptul că valorile hash sunt distribuite uniform pe întreg spațiul de ieșire.

5.5 Flexibilitate

Având în vedere utilizarea construcției Sponge și a arborelui Merkle, funcția hash propusă admite o dimensiune a hash-ului final configurabilă. Din punct de vedere teoretic, dimensiunea hash-ului poate fi de lungime arbitrară: 128 biți, 192 biți, 256 biți etc. Numărul de runde intermediare variază în funcție de lungimea hash-ului final, deci pentru același mesaj, hash-urile de lungimi diferite vor fi complet diferite.

5.6 Eficiența computațională

Având în vedere că funcția hash propusă utilizează construcția Sponge și arborele Merkle, funcția poate fi ușor paralelizată. De asemenea, utilizarea operațiilor pe biți, a operațiilor algebrice elementare și a tipurilor de date primitive sporește semnificativ viteza funcției. Implementarea funcției hash propuse a fost realizată în două limbaje de programare diferite: Go și Rust. Ambele implementări au fost optimizate pentru performanță, implementarea în Rust obținând viteze cu 30-40% mai mari decât cea în Go. Funcția hash propusă a fost testată pe seturi de date de dimensiuni variabile, cu un număr de nuclee alocate variabile. Testele au fost efectuate pe trei sisteme diferite, cu configurații hardware diferite și cu sisteme de operare diferite:

- Mac cu procesor Apple M2 Pro, 16GB RAM și macOS 15.6
- Laptop cu procesor AMD Ryzen 7 8845HS, 32GB RAM și Ubuntu 24.04
- Desktop cu procesor Intel Xeon Platinum 8370C, 64GB RAM și Windows 11

Rezultatele obținute pe sistemul cu procesor Apple M2 Pro, exprimate în MB/s, sunt prezentate în tabelul de mai jos:

Dimensiune (MB)	1 nucleu	2 nuclee	4 nuclee	8 nuclee	10 nuclee
0.25	52.65	155.38	268.55	338.79	343.48
1	91.49	173.59	271.40	421.63	519.94
4	89.51	174.95	338.89	162.70	570.02
16	92.27	178.69	342.54	560.61	627.20
64	93.11	166.53	327.01	582.62	639.29
256	90.96	180.47	331.52	571.27	661.22

Tabela 5: Viteza de procesare (MB/s) în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul Apple M2 Pro

Dimensiune (MB)	1 nucleu	2 nuclee	4 nuclee	8 nuclee	16 nuclee
0.25	67.09	188.10	324.59	468.86	864.34
1	131.49	209.21	379.07	595.84	831.97
4	135.54	248.40	428.89	779.25	955.24
16	133.67	246.30	415.68	637.99	938.65
64	138.62	259.55	445.26	714.29	966.03
256	139.12	255.76	446.70	741.35	970.63

Tabela 6: Viteza de procesare (MB/s) în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul AMD Ryzen 7 8845HS

Dimensiune (MB)	1 nucleu	2 nuclee	4 nuclee	8 nuclee	16 nuclee
0.25	79.90	140.52	201.08	78.50	188.41
1	84.53	137.24	281.44	358.55	356.11
4	82.88	143.51	284.30	405.09	549.16
16	82.40	153.20	300.18	497.98	562.38
64	71.85	158.33	208.55	494.77	641.86
256	85.73	168.38	305.16	511.90	756.98

Tabela 7: Viteza de procesare (MB/s) în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul Intel Xeon Platinum 8370C

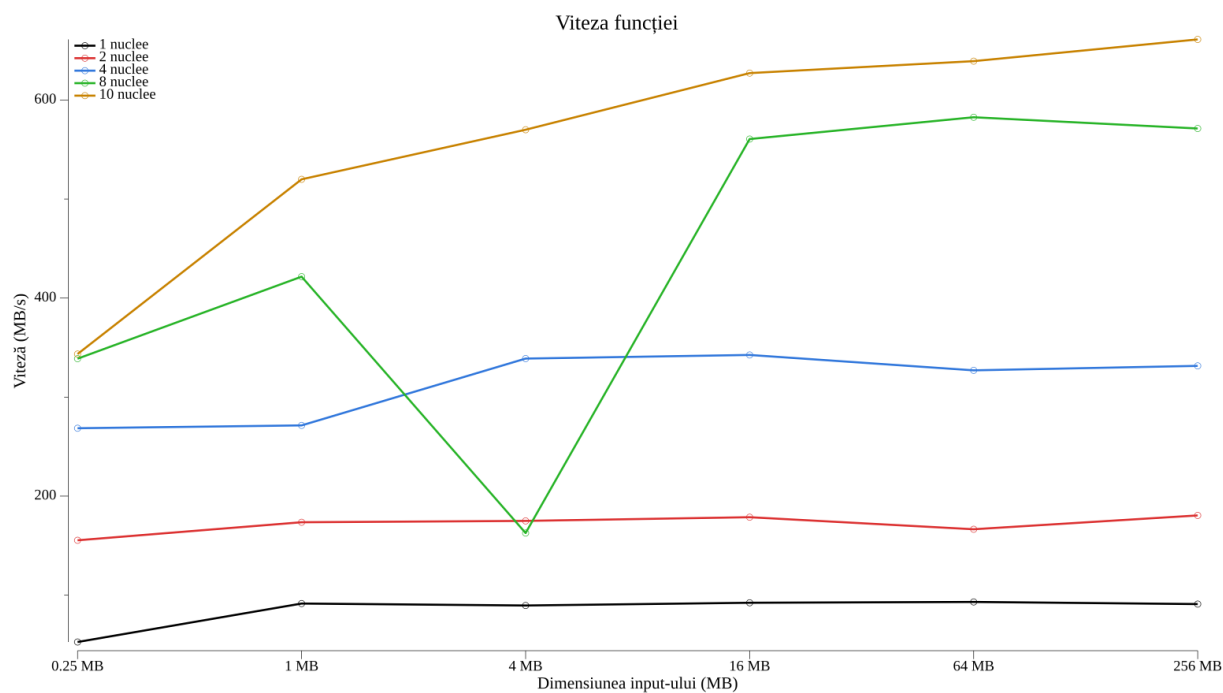


Figura 12: Viteza de procesare în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul Apple M2 Pro

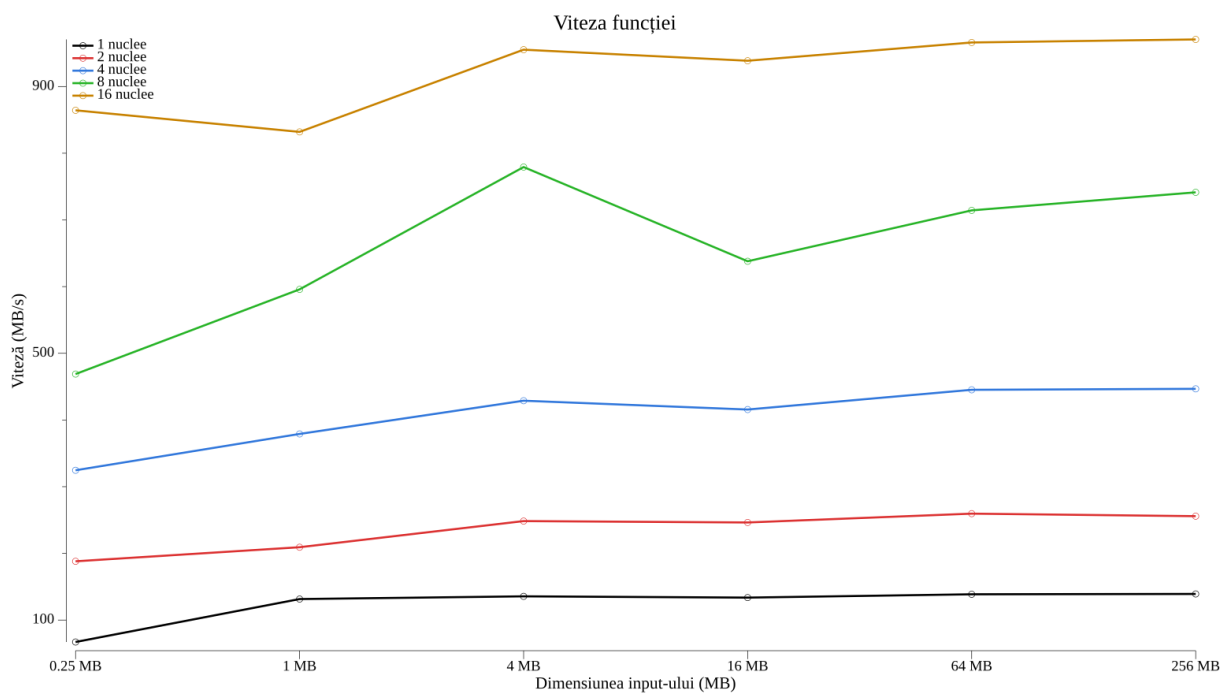


Figura 13: Viteza de procesare în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul AMD Ryzen 7 8845HS

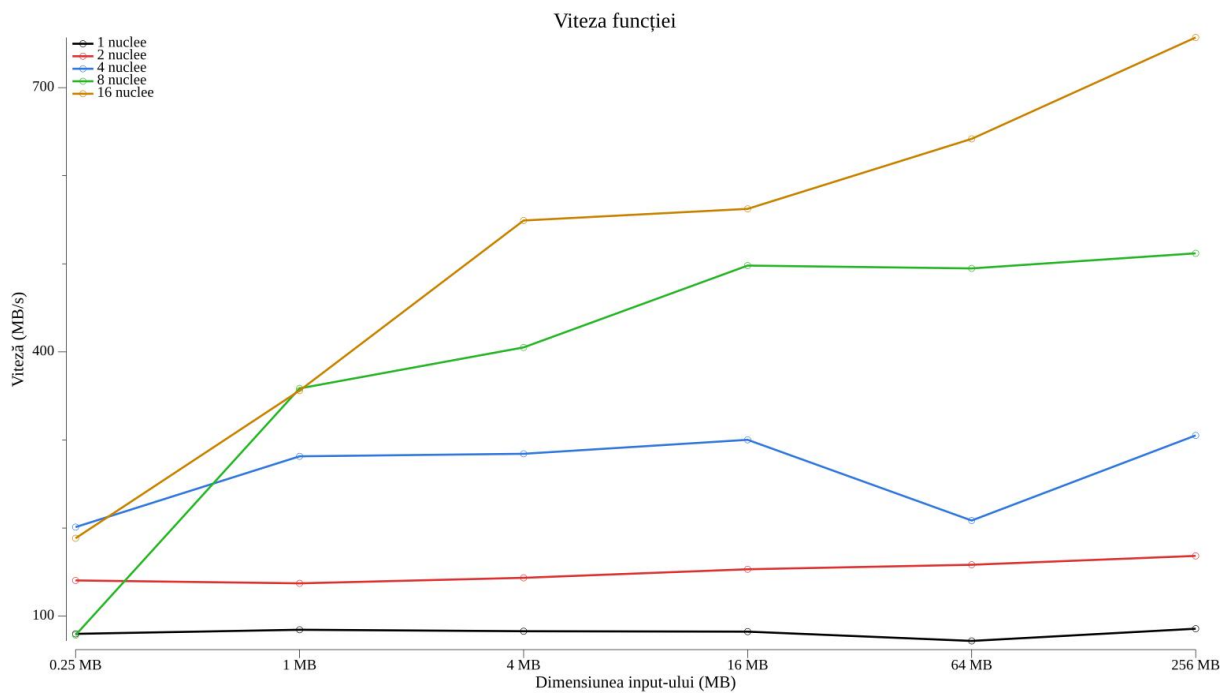


Figura 14: Viteza de procesare în funcție de dimensiunea datelor și de numărul de nuclee, pe procesorul Intel Xeon Platinum 8370C

Așadar, putem observa că pe majoritatea sistemelor moderne, funcția hash propusă va atinge viteze de procesare de ordinul a sute de MB/s. În toate comparațiile ulterioare vom considera că viteza funcției hash pe un sistem cu 8 nuclee este aceea de 600MB/s. Această viteză este comparabilă cu cea a funcțiilor hash utilizate la nivel de industrie, precum SHA-256 sau MD5. În ciuda faptului că aceste funcții hash consacrate rulează pe un singur nucleu și adesea permit optimizări hardware, funcția hash propusă atinge viteze similare. Acest rezultat se datorează în mare parte arborelui Merkle, care permite o paralelizare eficientă a întregului proces de calcul al hash-ului.

5.7 Comparația cu alte funcții hash

Pentru a evalua calitatea funcției hash propuse în raport cu literatura de specialitate, se vor compara toate metricile de performanță prezentate cu cele raportate de alte funcții hash bazate pe sisteme dinamice haotice. Toate valorile se referă la funcții hash pe 128 de biți și au fost determinate, acolo unde a fost posibil, pentru $N = 10000$ de mesaje de test. În tabelul 8 sunt prezentați cei șase indicatori statistici discutați anterior: numărul mediu de biți modificați B_{med} , numărul minim B_{min} și maxim B_{max} de biți modificați, probabilitatea medie de modificare P , precum și abaterile standard ΔB și ΔP .

Funcția hash	B_{med}	B_{min}	B_{max}	P (%)	ΔB	ΔP (%)
Funcția propusă	64.0771	42	85	50.0602	5.6502	4.4142
Amin et al. [3]	63.84	—	—	49.88	5.58	4.38
Kanso & Ghebleh [10]	63.94	45	84	49.95	5.64	4.41
Kanso & Ghebleh [11]	64.00	44	84	50.00	5.65	4.41
Wu & Wang [19]	64.014	—	—	50.011	5.654	4.417
Zhao et al. [21]	63.9279	40	85	49.94	5.3234	4.16
Li & Li [13]	64.0415	47	81	50.03	5.7929	4.53
Ahmad et al. [1]	63.781	45	83	49.828	5.937	4.638
Boriga & Tilcă [7]	64.2136	47	84	50.3315	5.7040	4.3421
Zaouia et al. [20]	64.005	43	84	50.004	5.582	4.361
Valoarea ideală	64	—	—	50	—	—

Tabela 8: Indicatorii de confuzie și difuzie din diverse funcții hash bazate pe sisteme dinamice haotice

În tabelul 11 este comparată viteza de procesare a funcției hash propuse cu cele raportate de alte funcții hash bazate pe sisteme dinamice haotice. Pentru funcția propusă sunt prezentate șase valori: trei corespunzătoare execuției pe câte 8 nuclee și trei cores-

Funcția hash	Minim	Maxim	Medie	Medie/byte
Funcția propusă	567	2278	1364.9053	85.3066
Kanso & Ghebleh [10]	656	2391	1364	85.25
Kanso & Ghebleh [11]	628	2326	1363.7	85.23
Li & Li [13]	685	1972	1253	78.31
Ahmad et al. [1]	796	2436	1374.1	85.88
Zaouia et al. [20]	520	2256	1365.03	85.31
Valoarea ideală	—	—	1360	85.33

Tabela 9: Suma diferenței absolute a bytes-ilor în diverse funcții hash bazate pe sisteme dinamice haotice

Funcția hash	0	1	2	3	4	5
Funcția propusă	9401	589	10	0	0	0
Kanso & Ghebleh [11]	9403	586	11	0	0	0
Li & Li [13]	9495	505	0	0	0	0
Ahmad et al. [1]	9387	586	27	0	0	0
Valori teoretice	9393	590	17	0	0	0

Tabela 10: Distribuția numărului de bytes egali în diverse funcții hash bazate pe sisteme dinamice haotice

punzătoare vitezei maxime obținute pe un singur nucleu, câte una pentru fiecare dintre cele trei sisteme de testare. Pentru a permite o comparație unitară, toate valorile au fost exprimate în MB/s; acolo unde sursele raportează viteza în Mb/s (megabiți pe secundă), aceasta a fost convertită prin împărțire la 8. Funcțiile hash care nu raportează o viteză de procesare comparabilă nu au fost incluse în comparație.

Funcția hash	Viteza (MB/s)	Hardware utilizat
Funcția propusă	571.27	Apple M2 Pro, 8 nuclee
Funcția propusă	741.35	AMD Ryzen 7 8845HS, 8 nuclee
Funcția propusă	511.90	Intel Xeon Platinum 8370C, 8 nuclee
Funcția propusă	93.11	Apple M2 Pro, 1 nucleu
Funcția propusă	139.12	AMD Ryzen 7 8845HS, 1 nucleu
Funcția propusă	85.73	Intel Xeon Platinum 8370C, 1 nucleu
Kanso & Ghebleh [10]	8.75	Intel Core i7 @ 2.70 GHz, 1 nucleu
Kanso & Ghebleh [11]	10.31	Intel Core i7-2600 @ 3.40 GHz, 1 nucleu
Li & Li [13]	16.51	Intel Pentium IV @ 2.50 GHz, 1 nucleu
Kwok & Tang [12]	63.77	Intel Pentium IV @ 2.40 GHz, 1 nucleu
Boriga & Tilcă [7]	72.00	Intel Core i3 @ 2.53 GHz, 1 nucleu
Ahmad et al. [1]	116.65	Intel Core i5-6300U @ 2.40 GHz, 1 nucleu

Tabela 11: Compararea vitezei de procesare a funcției hash propuse cu cea raportată de alte funcții hash bazate pe sisteme dinamice haotice

Se observă că funcția hash propusă atinge viteze de procesare semnificativ mai mari

decât cele raportate de funcțiile hash din literatura de specialitate. Trebuie menționat însă, că funcția hash a fost testată pe sisteme hardware moderne, iar discrepanța de performanță poate fi influențată și de hardware-ul utilizat. În plus, proprietățile de confuzie și difuzie, rezistența la coliziuni și distribuția valorilor hash a funcției propuse sunt comparabile sau chiar superioare celor raportate de alte funcții hash bazate pe sisteme dinamice haotice. Astfel, putem afirma ca funcția hash propusă constituie un instrument criptografic robust și eficient, utilizabil la nivel de industrie.

6 Concluzii

În această lucrare am proiectat și implementat o funcție hash bazată pe sisteme dinamice discrete haotice, construită în jurul structurii Sponge și al arborelui Merkle. Analiza statistică prezentată în capitolul anterior confirmă faptul că funcția propusă posedă proprietăți criptografice solide. Testele de confuzie și difuzie arată că o modificare minimă a mesajului de intrare determină o schimbare semnificativă a hash-ului. Rezistența la coliziuni este, de asemenea, foarte bună: distribuția numărului de bytes egali și suma diferențelor absolute ale bytes-ilor se apropie de valorile teoretice, iar numărul de coliziuni observate este sub limita teoretică. În plus, valorile hash sunt distribuite uniform pe întreg spațiul de ieșire, ceea ce împiedică atacurile bazate pe criptanaliză statistică. Toate aceste rezultate sunt comparabile cu cele raportate de funcțiile hash consacrate din literatura de specialitate, ceea ce indică un nivel de securitate adecvat aplicațiilor criptografice moderne.

Pe lângă proprietățile criptografice, funcția hash propusă se remarcă prin viteza și eficiența sa pe sistemele cu mai multe nuclee. Datorită arborelui Merkle, calculul hash-ului poate fi paralelizat eficient, viteza de procesare crescând aproape liniar cu numărul de nuclee alocate. Pe sistemele moderne testate, funcția atinge viteze de procesare de ordinul sutelor de MB/s, comparabile cu cele ale funcțiilor hash utilizate la nivel de industrie, precum SHA-256 sau MD5. Acest lucru este cu atât mai semnificativ cu cât funcțiile hash consacrate rulează pe un singur nucleu și beneficiază adesea de optimizări hardware dedicate.

Ca direcții viitoare de dezvoltare, o primă posibilitate ar fi implementarea funcției în limbajul C, în vederea obținerii unor viteze de procesare și mai mari. O a doua direcție o constituie explorarea metodelor de optimizare hardware, prin care funcția ar putea fi accelerată suplimentar pe arhitecturi specializate. În cele din urmă, o direcție promițătoare ar fi extinderea funcției hash într-o schemă completă de criptare și decriptare, valorificând proprietățile sistemelor dinamice haotice care stau la baza ei.

Referințe

- [1] Musheer Ahmad, Shruti Khurana, Sushmita Singh și Hamed D. AlSharari, „A Simple Secure Hash Function Scheme Using Multiple Chaotic Maps”, în *3D Research* 8.2 (2017), p. 13, DOI: [10.1007/s13319-017-0123-1](https://doi.org/10.1007/s13319-017-0123-1).
- [2] Gonzalo Alvarez și Shujun Li, „Some Basic Cryptographic Requirements for Chaos-Based Cryptosystems”, în *International Journal of Bifurcation and Chaos* 16.8 (2006), pp. 2129–2151, DOI: [10.1142/S0218127406015970](https://doi.org/10.1142/S0218127406015970).
- [3] Mohamed Amin, Osama S. Faragallah și Ahmed A. Abd El-Latif, „Chaos-based hash function (CBHF) for cryptographic applications”, în *Chaos, Solitons & Fractals* 42.2 (2009), pp. 767–772, DOI: [10.1016/j.chaos.2009.02.001](https://doi.org/10.1016/j.chaos.2009.02.001).
- [4] John Banks, Jeff Brooks, Grant Cairns, Gary Davis și Peter Stacey, „On Devaney’s Definition of Chaos”, în *The American Mathematical Monthly* 99.4 (1992), pp. 332–334, DOI: [10.1080/00029890.1992.11995856](https://doi.org/10.1080/00029890.1992.11995856).
- [5] Giancarlo Benettin, Luigi Galgani, Antonio Giorgilli și Jean-Marie Strelcyn, „Lyapunov Characteristic Exponents for Smooth Dynamical Systems and for Hamiltonian Systems; a Method for Computing All of Them. Part 1: Theory”, în *Meccanica* 15.1 (1980), pp. 9–20, DOI: [10.1007/BF02128236](https://doi.org/10.1007/BF02128236).
- [6] Guido Bertoni, Joan Daemen, Michaël Peeters și Gilles Van Assche, „Sponge Functions”, în *ECRYPT Hash Workshop*, 2007.
- [7] Radu Boriga și Marius Tilcă, „A New Fast and Secure Chaos-Based Keyed Hash Function”, în *U.P.B. Scientific Bulletin, Series C* 79.2 (2017).
- [8] Ivan Bjerre Damgård, „A Design Principle for Hash Functions”, în *Advances in Cryptology — CRYPTO ’89*, vol. 435, Lecture Notes in Computer Science, New York, NY: Springer, 1990, pp. 416–427, DOI: [10.1007/0-387-34805-0_39](https://doi.org/10.1007/0-387-34805-0_39).
- [9] Robert L. Devaney, *An Introduction to Chaotic Dynamical Systems*, a 2-a ed., Redwood City, CA: Addison-Wesley, 1989, ISBN: 9780201130461.

- [10] A. Kanso și M. Ghebleh, „A fast and efficient chaos-based keyed hash function”, în *Communications in Nonlinear Science and Numerical Simulation* 18.1 (2013), pp. 109–123, DOI: [10.1016/j.cnsns.2012.06.019](https://doi.org/10.1016/j.cnsns.2012.06.019).
- [11] A. Kanso și M. Ghebleh, „A structure-based chaotic hashing scheme”, în *Nonlinear Dynamics* 81.1–2 (2015), pp. 27–40, DOI: [10.1007/s11071-015-1970-z](https://doi.org/10.1007/s11071-015-1970-z).
- [12] H. S. Kwok și Wallace K. S. Tang, „A Chaos-Based Cryptographic Hash Function for Message Authentication”, în *International Journal of Bifurcation and Chaos* 15.12 (2005), pp. 4043–4050, DOI: [10.1142/S0218127405014489](https://doi.org/10.1142/S0218127405014489).
- [13] Yantao Li și Xiang Li, „Chaotic hash function based on circular shifts with variable parameters”, în *Chaos, Solitons and Fractals* 91 (2016), pp. 639–648, DOI: [10.1016/j.chaos.2016.08.014](https://doi.org/10.1016/j.chaos.2016.08.014).
- [14] Ralph C. Merkle, „One Way Hash Functions and DES”, în *Advances in Cryptology — CRYPTO '89*, vol. 435, Lecture Notes in Computer Science, New York, NY: Springer, 1990, pp. 428–446, DOI: [10.1007/0-387-34805-0_40](https://doi.org/10.1007/0-387-34805-0_40).
- [15] National Institute of Standards and Technology, Secure Hash Standard (SHS), Federal Information Processing Standards Publication FIPS PUB 180-1, National Institute of Standards și Technology, 1995.
- [16] National Institute of Standards and Technology, Secure Hash Standard (SHS), Federal Information Processing Standards Publication FIPS PUB 180-4, National Institute of Standards și Technology, 2015, DOI: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4).
- [17] Ronald L. Rivest, The MD5 Message-Digest Algorithm, RFC 1321, Internet Engineering Task Force, 1992, DOI: [10.17487/RFC1321](https://doi.org/10.17487/RFC1321).
- [18] Claude E. Shannon, „Communication Theory of Secrecy Systems”, în *The Bell System Technical Journal* 28.4 (1949), pp. 656–715, DOI: [10.1002/j.1538-7305.1949.tb00928.x](https://doi.org/10.1002/j.1538-7305.1949.tb00928.x).
- [19] Di Wu și Xiaomin Wang, „A Chaos-Based Hash Function”, în *Proceedings of the International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)* (2015).

- [20] Nassim Zaouia, Kheyreddine Djouzi și Mehammed Daoui, „Collisions-Resistant Hash Function Based on a Logistics Map”, în Proceedings of the International Conference on Advances in Electronics, Control and Communication Systems, 2023.
- [21] Changwei Zhao et al., „One-Way Hash Function based on Cascade Chaos”, în The Open Cybernetics & Systemics Journal 9 (2015), pp. 573–580, DOI: [10.2174/1874110X01509010573](https://doi.org/10.2174/1874110X01509010573).